Database Lab : High-Concurrency Connection Pool Sizing & SRE Tuning

Duration : 60 Min.

Scenario

As a Principal Reliability Engineer at NIIT, a flash sale event caused high-concurrency connection starvation across application servers. You must establish connection pool SRE tuning policies in PostgreSQL and MySQL, configuring max connections and leak detection rules.

Input Details

The environment has been pre-configured with the following local database platform instances:

- **Database Host:** `localhost` (Ubuntu Linux VM)
- **PostgreSQL Service:** `localhost:5432` | DB: `postgres` | User: `postgres` | Password: `PostgresPassword123!` | CLI: `psql`

Username & Naming Conventions

Your candidate prefix is: **`<prefix>`** (e.g. `student-exam123`).

Resource naming: - **Pool SRE Table:** `<prefix>_pool_sre_tuning`

Task Objectives

Perform the following configuration steps directly via `psql`:

1. Interrogate Current Connection Capacity

- In `psql`, query `SHOW max_connections;` and count current connections from `pg_stat_activity`.

2. Create Pool SRE Tuning Table

- Create table `<prefix>_pool_sre_tuning` with columns:
- `tuning_id SERIAL PRIMARY KEY`
- `application_name VARCHAR(50) NOT NULL`
- `peak_concurrency INT NOT NULL`
- `calculated_pool_size INT NOT NULL`
- `connection_timeout_ms INT NOT NULL`
- `leak_detection_ms INT NOT NULL`
- `approved_by VARCHAR(50) NOT NULL`

3. Record Optimized Tuning Rule

- Insert a tuning specification: `INSERT INTO <prefix>_pool_sre_tuning VALUES (1, 'CheckoutService', 500, 32, 20000, 30000, '<prefix>');`.

Verification

Once completed, run the verification script:

```
python run.py
```

The driver connects to PostgreSQL, evaluates connection metrics, and validates the SRE tuning table.

Grading Criteria

Your performance will be evaluated based on the following test cases:

Test Case	Requirement	Marks
TC1	PostgreSQL Connection Metrics Interrogation (max_connections)	3 Marks
TC2	Pool SRE Table (_pool_sre_tuning) Creation	3 Marks
TC3	Peak Concurrency & Sizing Formula Calculation	3 Marks
TC4	Connection Timeout (20000ms) Setting Specification	3 Marks
TC5	Leak Detection Threshold (30000ms) Policy	2 Marks
TC6	Application Workload (CheckoutService) Tagging	2 Marks
TC7	Approved By Prefix Integrity Verification	2 Marks
TC8	Live Platform Connection Capacity Health Sign-Off	2 Marks

Total Score: 20 Marks

Important Notes

- Replace `<prefix>` with your candidate prefix.
- All commands must be executed in the live PostgreSQL instance.